

COMP.SE.140 Exercise1 Report - Joona Syrjämäki H291526

Platform information

This exercise was developed using a both a Windows based PC and a Linux based laptop with the following specs:

①

Device specifications

Copy

^

Device name	DESKTOP-P862VPR	
Processor	AMD Ryzen 5 7600X 6-Core Processor	(4.70 GHz)
Installed RAM	32,0 GB (31,1 GB usable)	
Device ID	408521FE-D74E-436B-94C4-9B4F04B96B06	
Product ID	00326-10000-00000-AA876	
System type	64-bit operating system, x64-based processor	
Pen and touch	No pen or touch input is available for this display	

Related links

Domain or workgroup

System protection

Advanced system settings

☐

Windows specifications

Copy

^

Edition	Windows 11 Home	
Version	24H2	
Installed on	31/01/2025	
OS build	26100.6584	
Experience	Windows Feature Experience Pack 1000.26100.234.0	
Microsoft Services Agreement		
Microsoft Software License Terms		

^

🏠

~

ff

~`
`0+`
`ooo/
`+oooo:
`+oooooo:
-+oooooo+:
`/:-:+oooo+:
`/++++/+++++:
`/+++++++:
`/+++ooooooooooooo/`
./ooooosssso++osssssso+`
..osssssso-`/osssssso+`
-osssssso. :ssssssso.
:osssssso/ osssso+++.
/osssssso/ +sssssoo/-
`/osssssso+/- -:/+osssso+-
+sso+:` `.-/+oso:
++:.` `.-/+/
`/`

syrjis@matebook

OS: Arch Linux x86_64

Host: MACH-WX9 (M12)

Kernel: Linux 6.16.8-arch3-1

Uptime: 45 mins

Packages: 885 (pacman), 17 (flatpak)

Shell: zsh 5.9

Display (JDI422A): 3000x2000 @ 1.75x in 14", 60 Hz [Built-in]

DE: KDE Plasma 6.4.5

WM: KWin (Wayland)

WM Theme: Breeze

Theme: Breeze (Sweet) [Qt], Breeze-Dark [GTK2], Breeze [GTK3]

Icons: candy-icons [Qt], candy-icons [GTK2/3/4]

Font: GeistMono Nerd Font (10pt) [Qt], GeistMono Nerd Font (10pt) [GTK2/3/4]

Cursor: Sweet (24px)

Terminal: konsole 25.8.1

Terminal Font: GeistMono NF (12pt, Regular)

CPU: Intel(R) Core(TM) i7-8550U (8) @ 4.00 GHz

GPU 1: NVIDIA GeForce MX150 [Discrete]

GPU 2: Intel UHD Graphics 620 @ 1.15 GHz [Integrated]

Memory: 5.50 GiB / 15.37 GiB (36%)

Swap: 0 B / 4.00 GiB (0%)

Disk (/): 45.47 GiB / 467.40 GiB (10%) - ext4

Docker versions were:

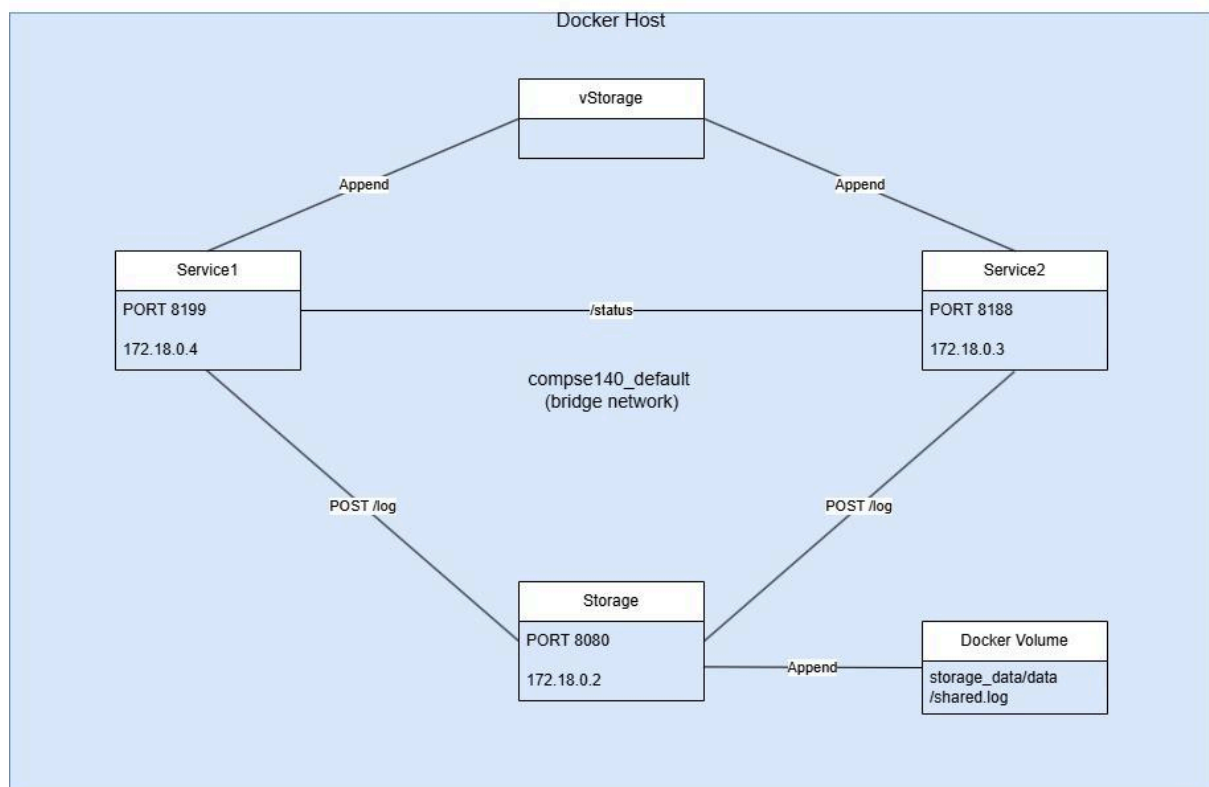
- Docker Desktop: 4.34.2 (167172)
- Docker Engine: v27.2.0
- Docker Compose: v2.29.2-desktop.2

Development tools used were:

- IntelliJ IDEA for Kotlin
- VS Code for Node.js and Docker related files
- Vim for other quick fixes

Services diagram

Below is a service diagram showcasing the services, network and storage. IP addresses of the services were spied using *docker network inspect compse140_default*



Measurements analysis

Uptime

Service1 (Node.js) was set to measure `process.uptime()` and Service2 (Kotlin) was set to measure `ManagementFactory.getRuntimeMxBean().uptime`. Essentially, both were measuring the time since the services were started inside the Docker container. This means if a container is stopped and restarted, uptime is reset to 0.

Initially, I set the services to track *proc/uptime* from system files, but it was quickly apparent that this was actually computing how long the VM that Docker runs on had been running. This was rather irrelevant information in this case (but might actually be useful in a real system to measure host uptime) so I changed to measure uptime since processes started.

In retrospect, I think the best thing would be to measure container uptime.

Disc Space

In Service1 (Node.js) I used the `check-disk-space` package to check free space from path `"/`". In Service2 (Kotlin), I used `FileSystems.getDefault().fileStores.first().usableSpace`, which also reports free space of the container's root file system (meaning path `"/`" as well).

This means that we are tracking and reporting free space inside the overlay filesystem, which is basically the host's disk space available to Docker. In my case, this was around 900 GB with a 1 TB HDD. Appending log lines barely had an effect on this number as they are just fractions of a megabyte.

In terms of disk space I am not sure what the best practice/better option would be. I think more relevant tracking would be to bind the measuring to the volumes some how(?).

Storage systems

vStorage

The good thing about the vStorage solution was that it is very easy to inspect and access even if the containers are not up and running. It is also very easy to understand and implement, as it is just a single file that gets appended to when services want to log something. Quite handy for quick debugging, but not at all suitable for production level code.

The disadvantages of this are that `./vstorage` can't be shared across different machines, it is very much at risk to manual editing/deletions, and it can also run into trouble when used on different operating systems as windows, macos, and linux all handle paths, line endings and permissions differently.

Storage service

The storage service in turn persists data better and more safely. Everything lives inside the `/data` directory that relates to a Docker volume `storage_data`. Volumes persist data if the container is restarted or rebuilt. Also, this version works consistently on any operating system since the location is abstracted.

The disadvantages of this is that this data is not as directly visible as a file in the repo. This also makes these logs harder to version control (if that is even something that is wanted). Having access to this is harder too as it requires Docker running and if the container is composed down with the `-v` flag, data is gone permanently unless backed up somehow.

Cleanup instructions

Logs can be cleared easily using the implemented `/clear` endpoint. This clears both the `vStorage` and the persistent storage (but requires the container to be running):

```
curl http://localhost:8199/clear
```

In the case of clearing logs with the containers stopped, `vStorage` can be cleared with:

```
: > ./vstorage (Linux/macOS)
Set-Content .\vstorage -Value "" (PowerShell)
```

And then using `docker compose down -v` resets everything inside Docker to also clear the persistent storage.

Difficulties & problems

I had the most difficulties with setting up `Service2`. I chose Kotlin as the second language I used because I started learning it recently, but I had a very hard time setting it up this time.

Also, I had a hard time grasping the idea of mounting a single file in a volume. To be honest, the whole structure and expected outcome of the storage system/combination was quite unclear from the instructions. I did my best but was very uncertain about everything I was doing.